Image filtering

- Modify the pixels in an image based on some function of a local neighborhood of each pixel

10	5	3
4	5	1
1	1	7

Local image data

Some function

	7	

Modified image data

Source: L. Zhang

Linear filtering

- One simple version of filtering: linear filtering (cross-correlation, convolution)
  - Replace each pixel by a linear combination (a weighted sum) of its neighbors
- The prescription for the linear combination is called the “kernel” (or “mask”, “filter”)

10	5	3
4	6	1
1	1	8

Local image data

0	0	0
0	0.5	0
0	1	0.5

kernel

	8	

Modified image data

Source: L. Zhang

Cross-correlation

Let F be the imag H be the kernel (of size 2*k*+1 x 2*k*+1) G and be the output image

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i+u,j+v]$$

This is called a **cross-correlation** oper

$$G = H \otimes F$$

- Can think of as a “dot product” between local neighborhood and kernel for each pixel

Convolution

- Same as cross-correlation, except that the kernel is “flipped” (horizontally and vertically)

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i-u,j-v]$$

This is called a **convolution** operation:

$$G = H * F$$

- Convolution is **commutative** and **associative**

*-F*G = G*F (commutative)*

- F(G*I) = (F*G)*I (associative)*

The machine learning framework

$$y = f(\mathbf{x})$$

output — prediction function — image / image feature

- **Training:** given a *training set* of labeled examples $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, estimate the prediction function f by minimizing the prediction error on the training set
- **Testing:** apply f to a never before seen *test example* $\mathbf{x}$ and output the predicted value $y = f(\mathbf{x})$

Slide credit: L. Lazebnik

The old-school way

Training

Training Images

Training Labels

Image Features → Training → Learned model

Testing

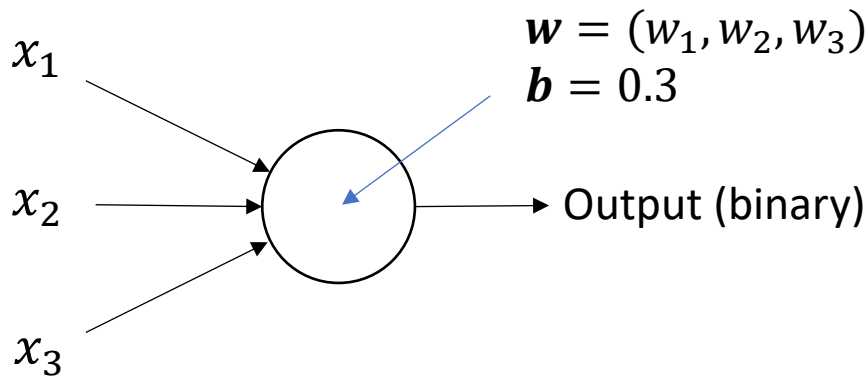
Test Image

Image Features → Learned model → Prediction

Slide credit: D. Hoiem and L. Lazebnik

Neural Networks

- Basic building block for composition is a *perceptron* (Rosenblatt c.1960)
- Linear classifier – vector of weights *w* and a 'bias' *b*



activation functions

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

$$w \cdot x \equiv \sum_j w_j x_j$$

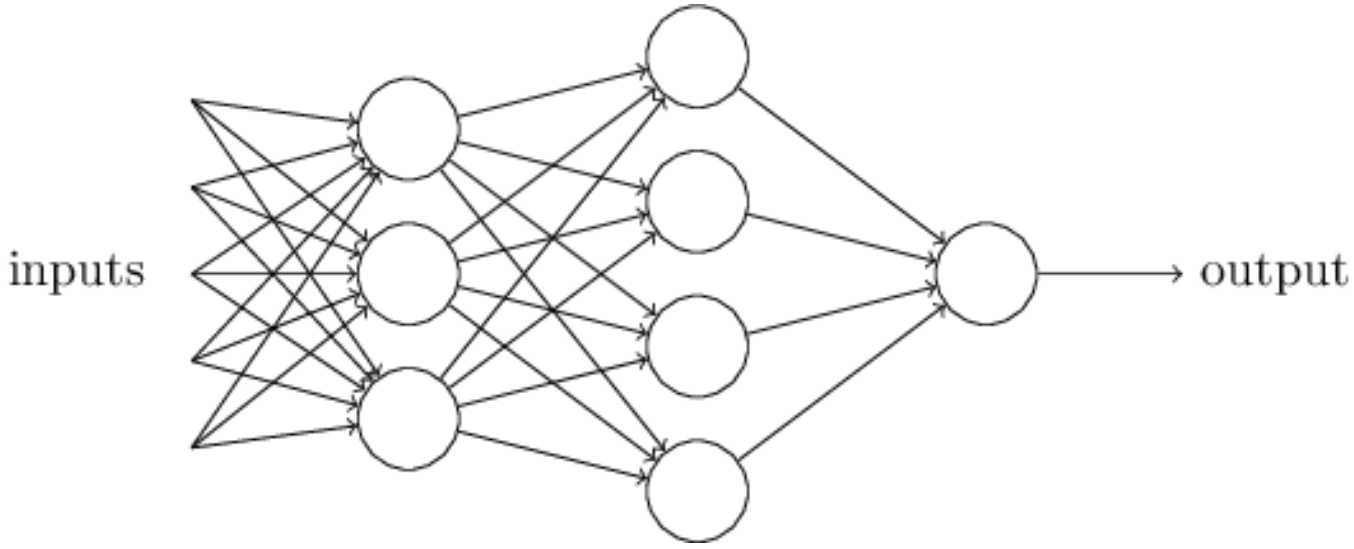
Binary classifying an image

- Each pixel of the image would be an input.
- So, for a 28 x 28 image, we vectorize.
- **x** = 1 x 784

- **w** is a vector of weights for each pixel, 784 x 1
- b is a scalar bias per perceptron

- result = **xw** + b -> (1x784) x (784x1) + b = (1x1)+b

Composition

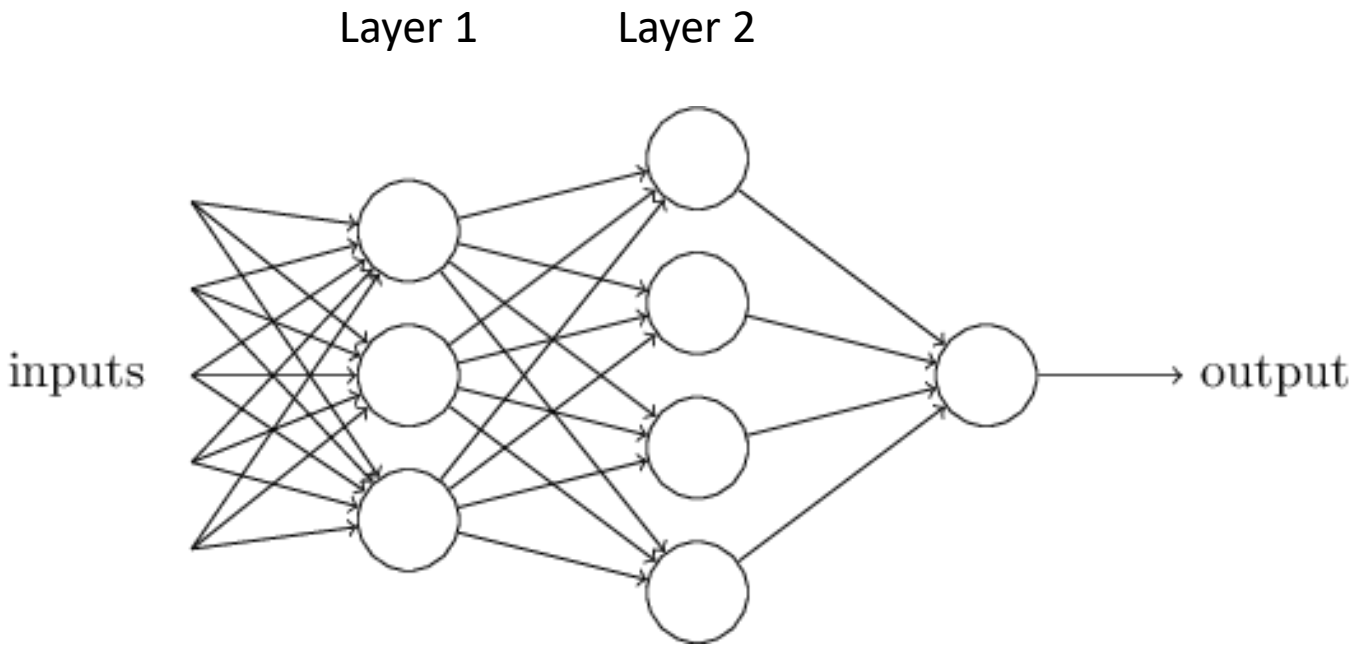


Attempt to represent complex functions as compositions of smaller functions.

Outputs from one perceptron are fed into inputs of another perceptron.

Nielsen

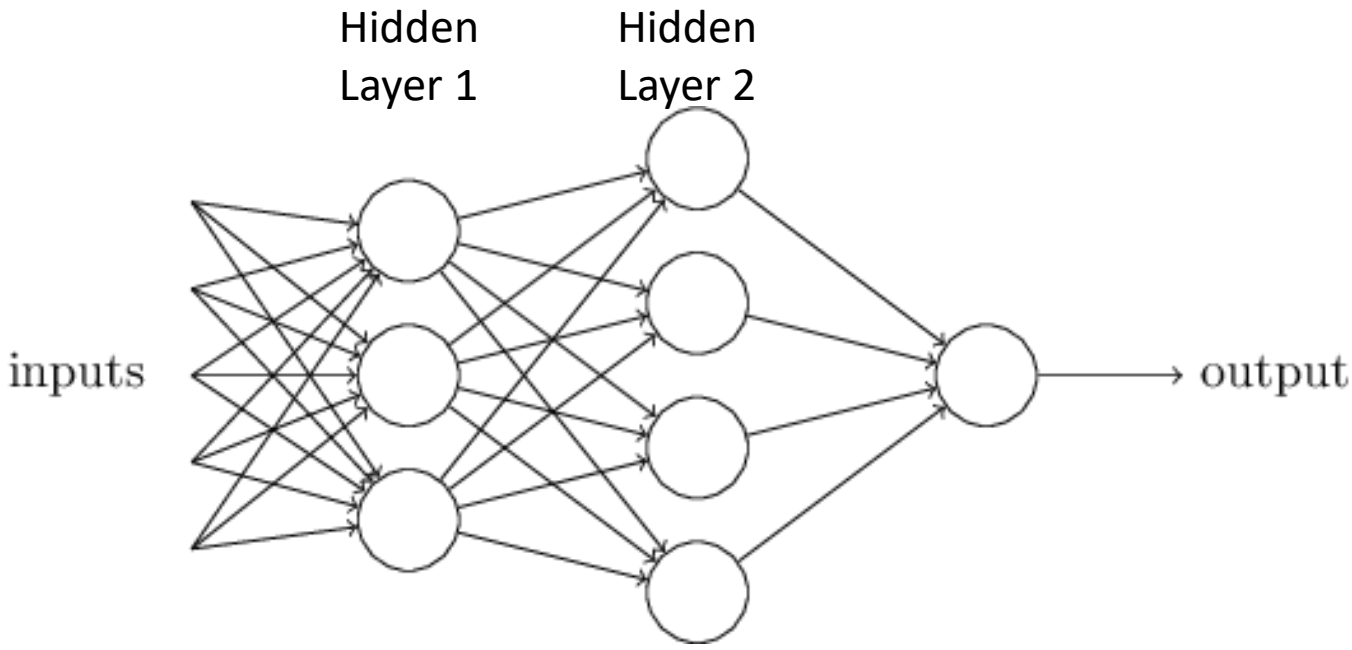
Composition



Sets of layers and the connections (weights) between them define the *network architecture.*

Nielsen

Composition



Layers that are in between the input and the output are called *hidden layers*, because we are going to *learn* their weights via an optimization process.

Nielsen

Linear functions

- We have formed chains of linear functions.
- We know that linear functions can be combined
  - g = f(h(x))

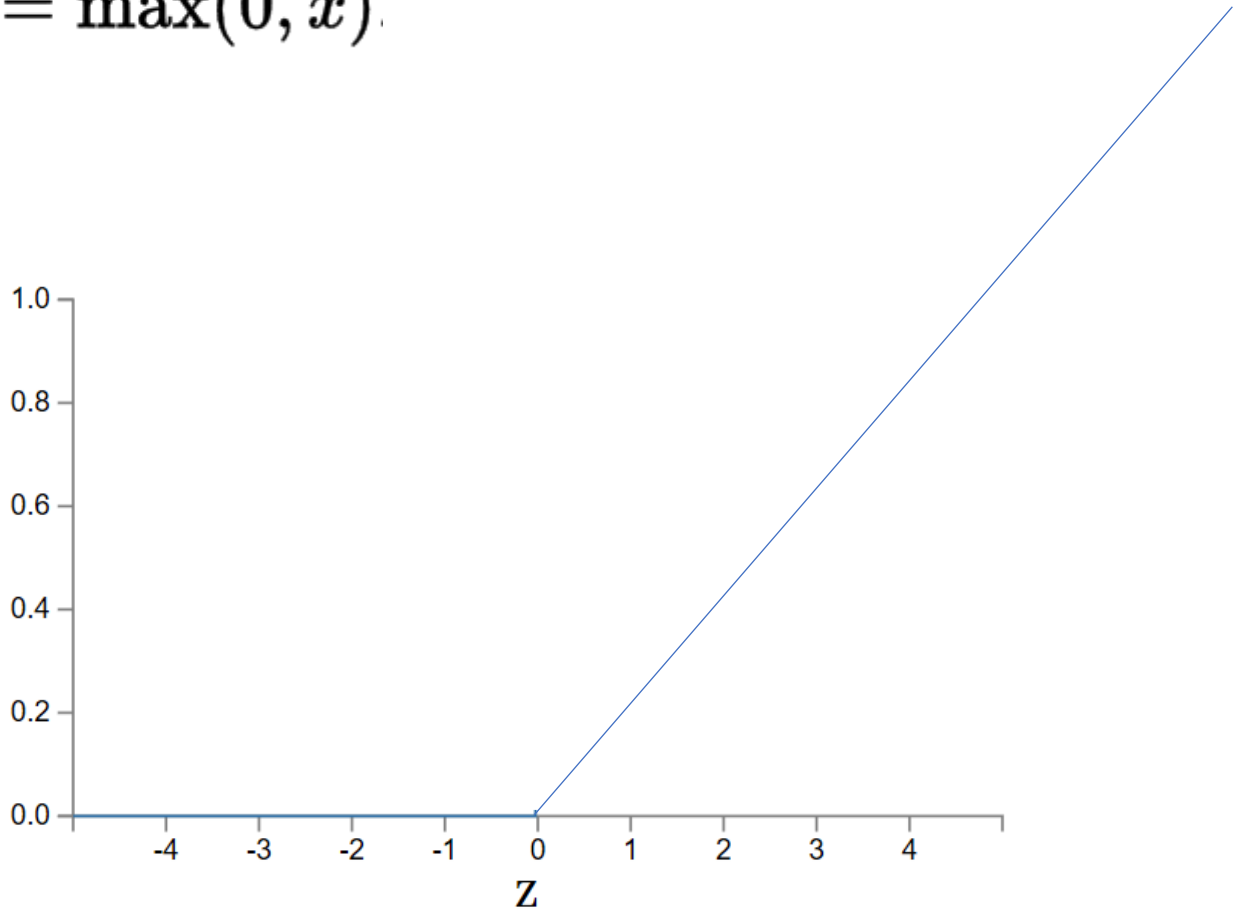
Our composition of functions is really just a single function

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

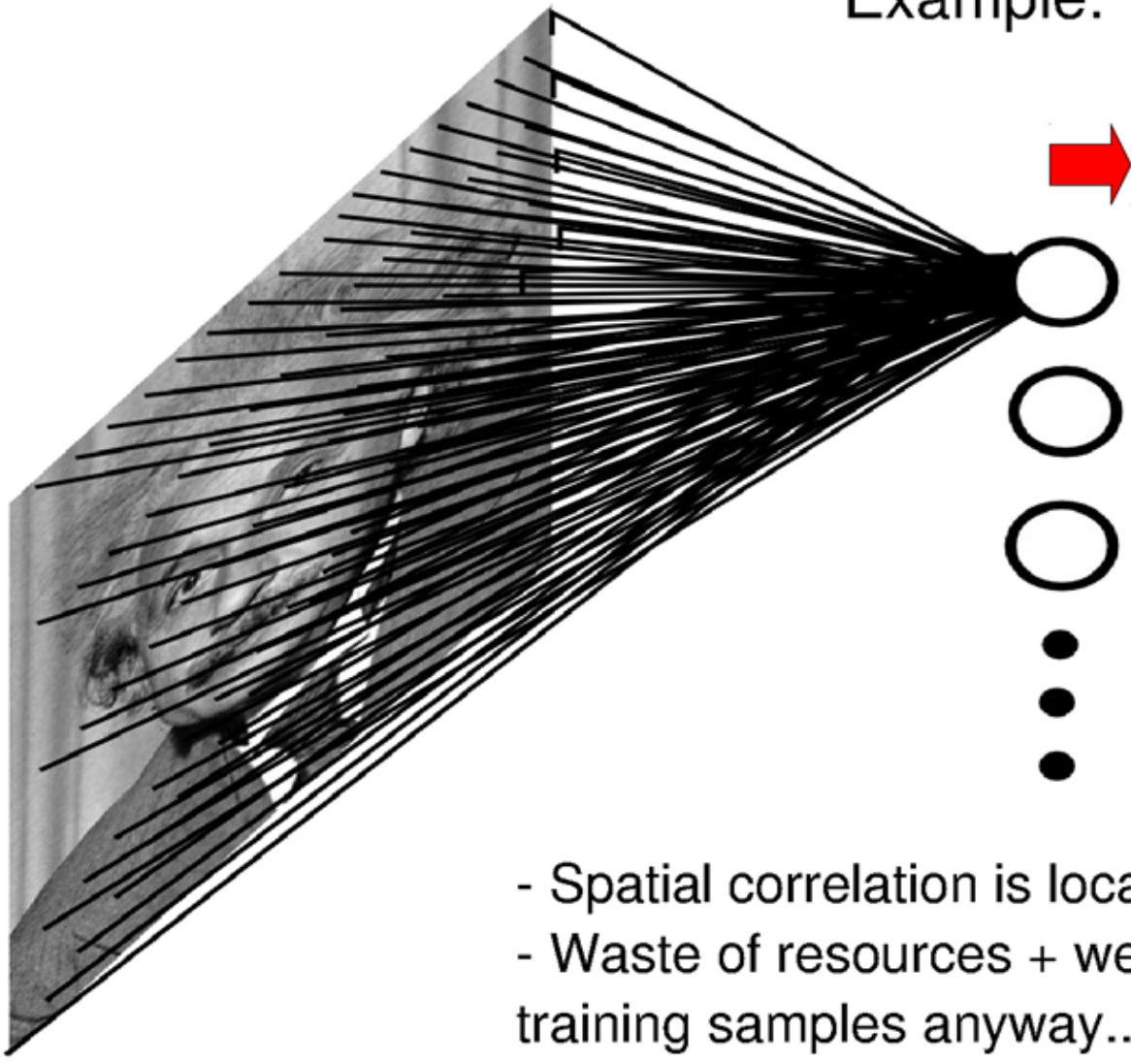
nonlinear

Rectified Linear Unit

- ReLU $$f(x) = \max(0, x)$$



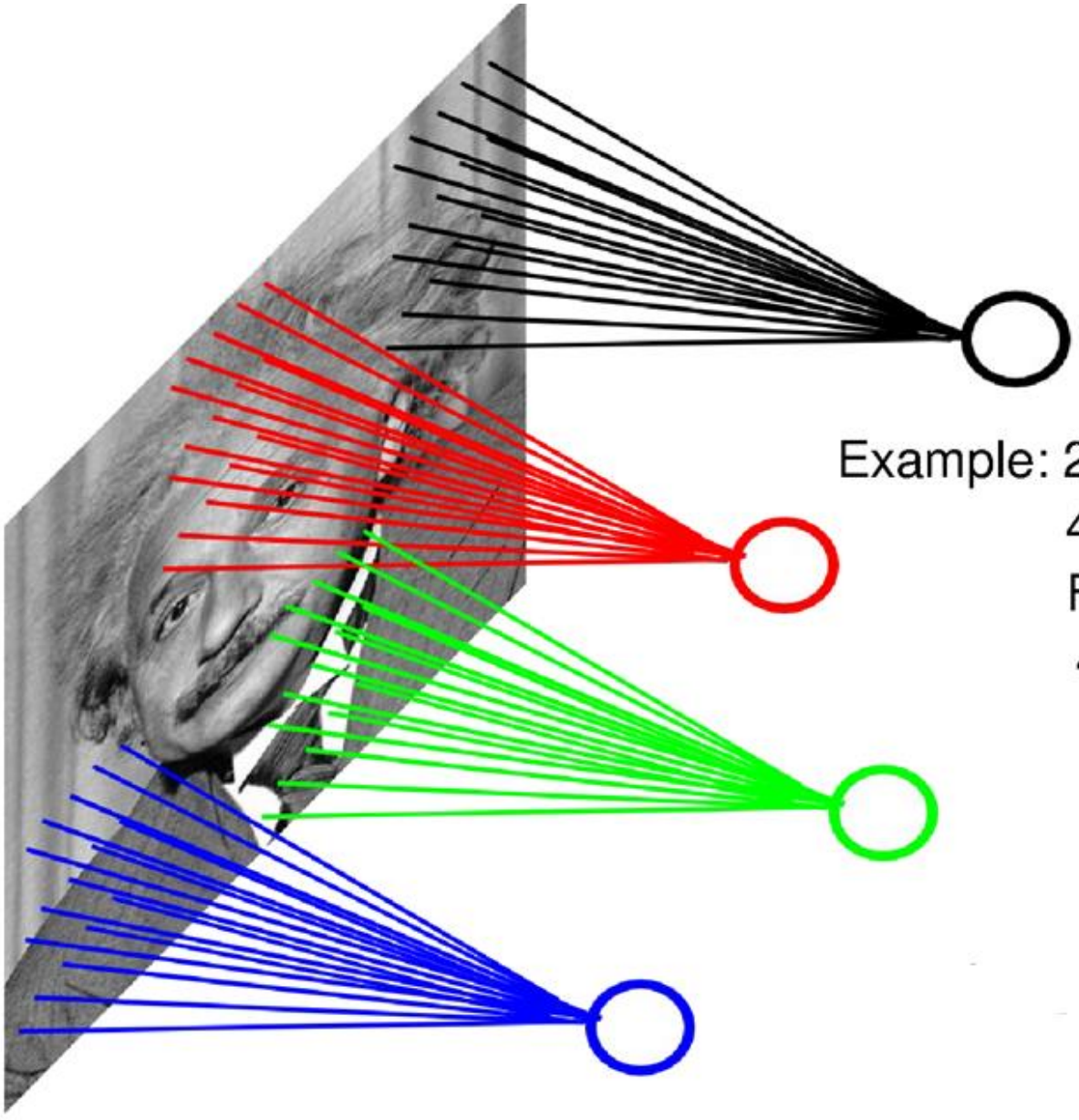
Images as input to neural networks



Example: 200x200 image
40K hidden units
~2B parameters!!!

- Spatial correlation is local
- Waste of resources + we have not enough training samples anyway..

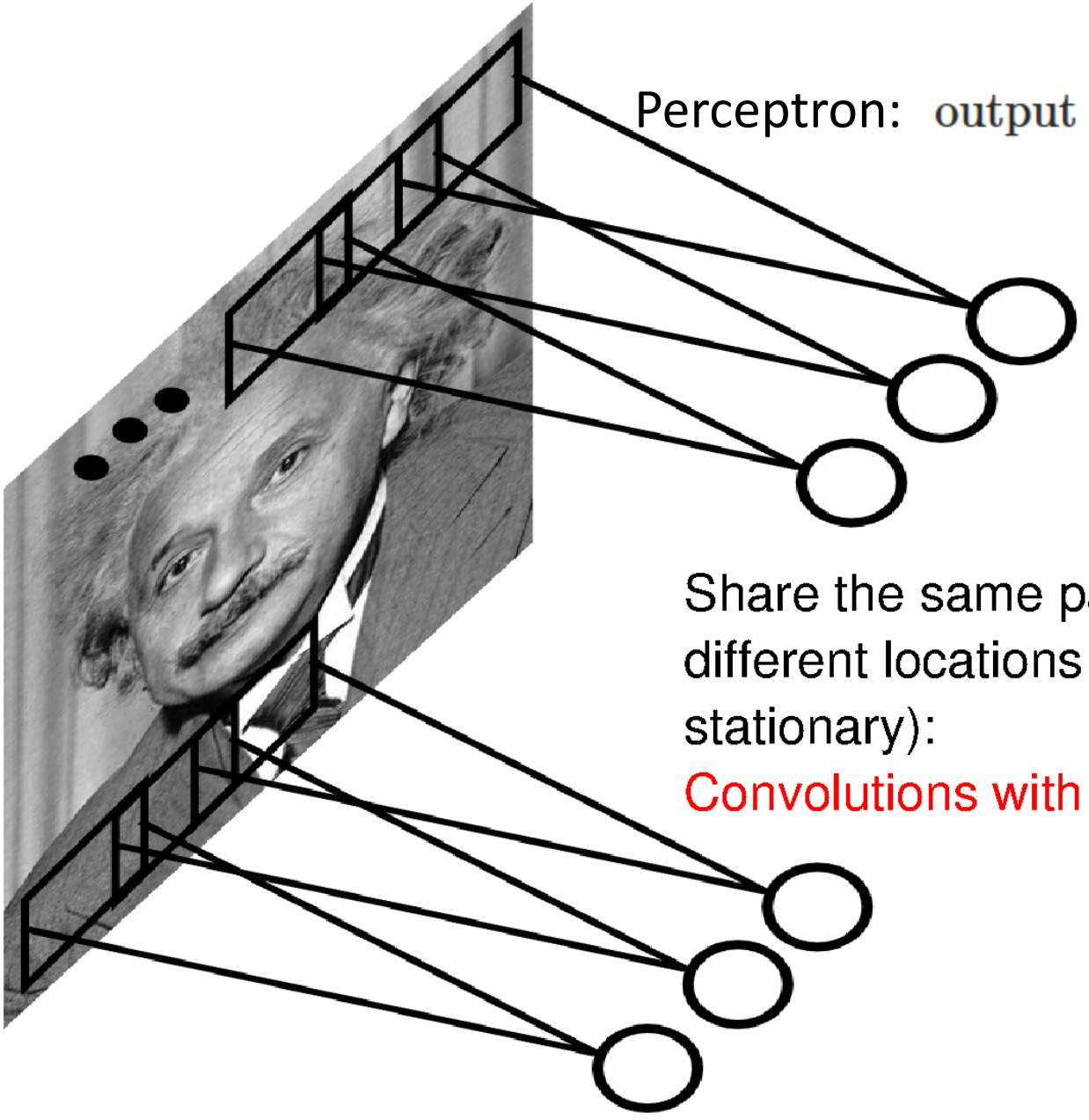
Ranzato



Example: 200x200 image
40K hidden units
Filter size: 10x10
4M parameters

Ranzato

Convolutional Layer



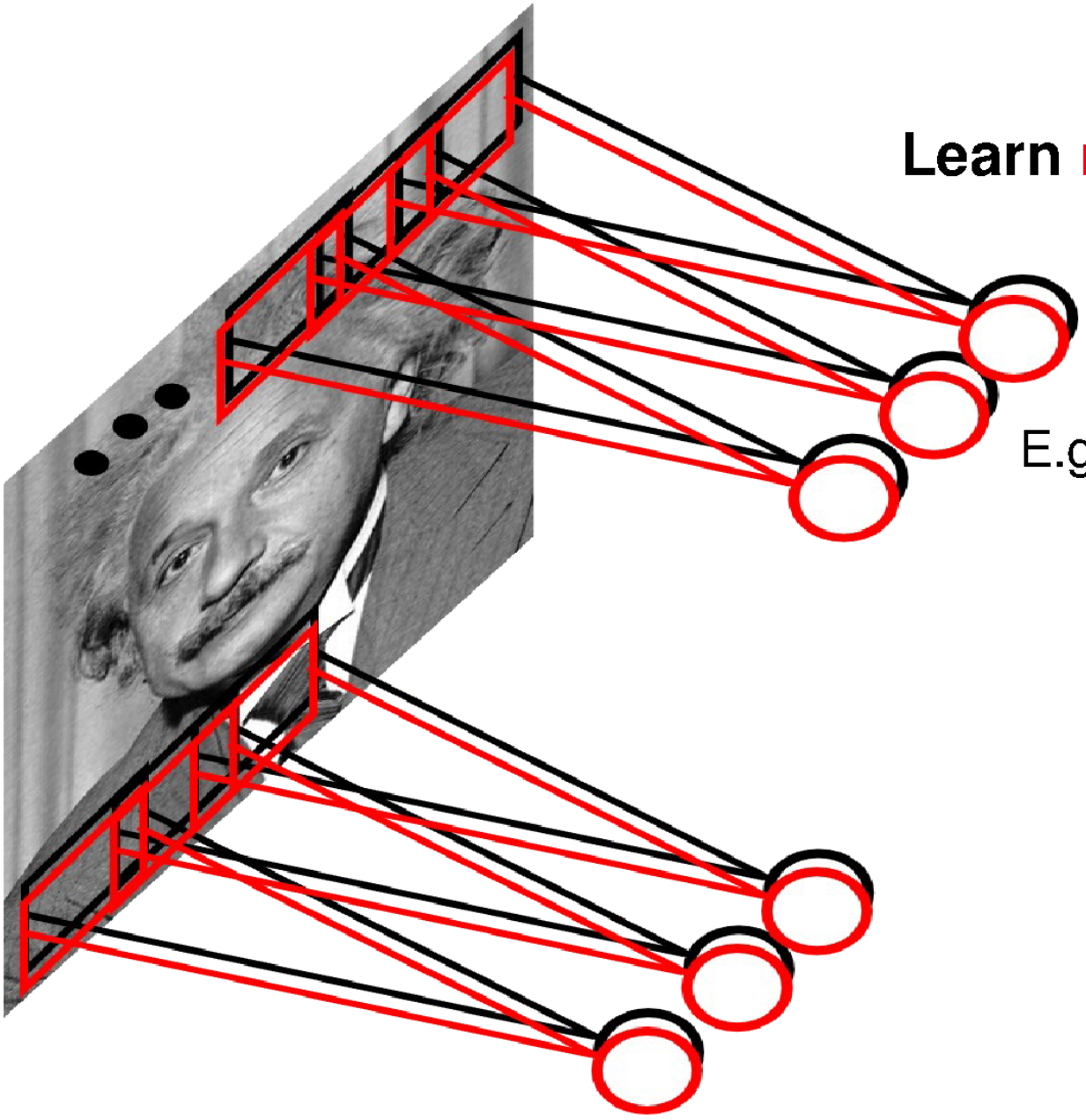
Perceptron: $\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$

$w \cdot x \equiv \sum_j w_j x_j$

Share the same parameters across different locations (assuming input is stationary):

Convolutions with learned kernels

Convolutional Layer

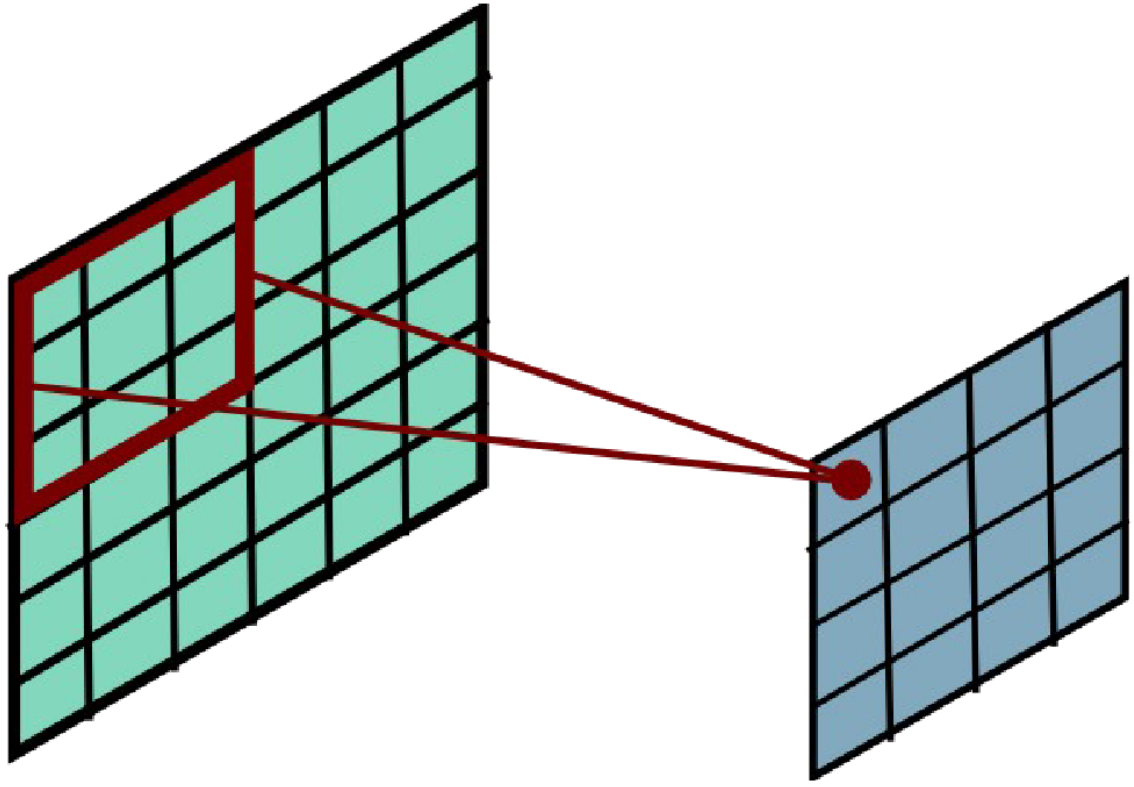


Learn multiple filters.

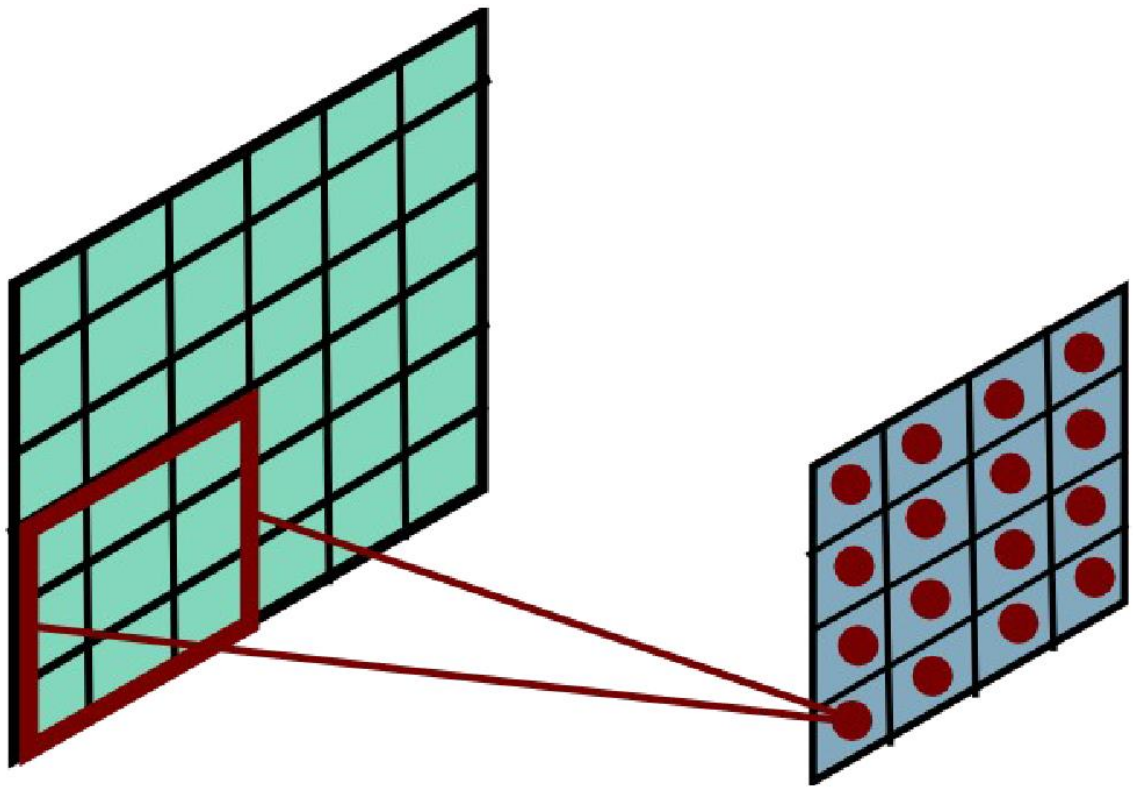
Filter = 'local' perceptron.
Also called *kernel.*

E.g.: 200x200 image
100 Filters
Filter size: 10x10
10K parameters

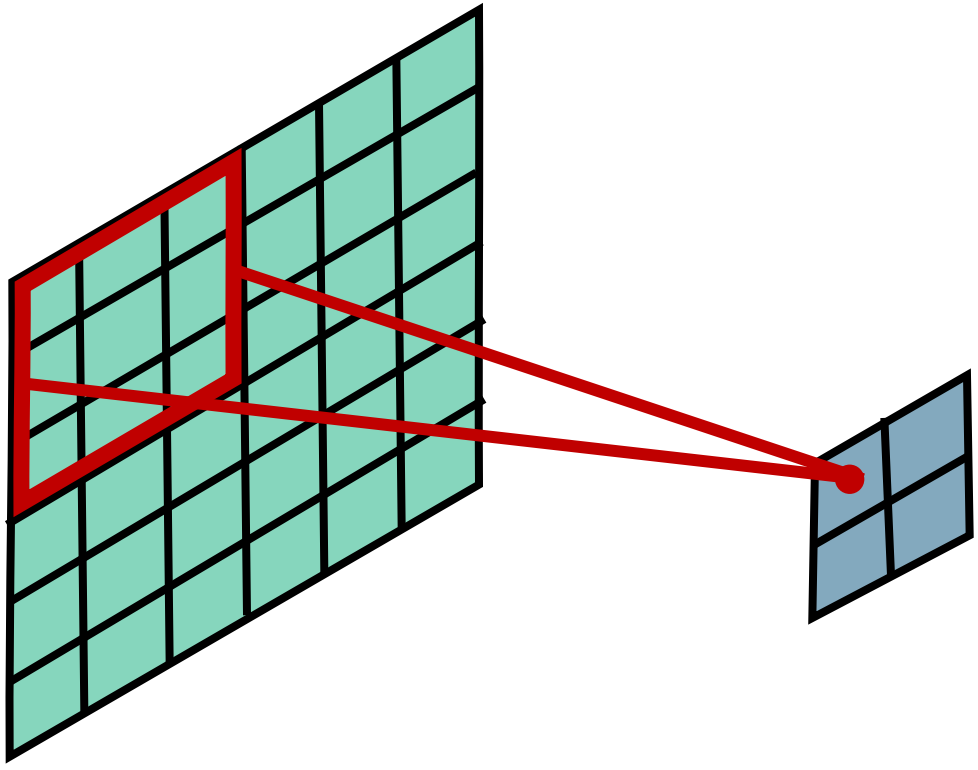
Stride = 1



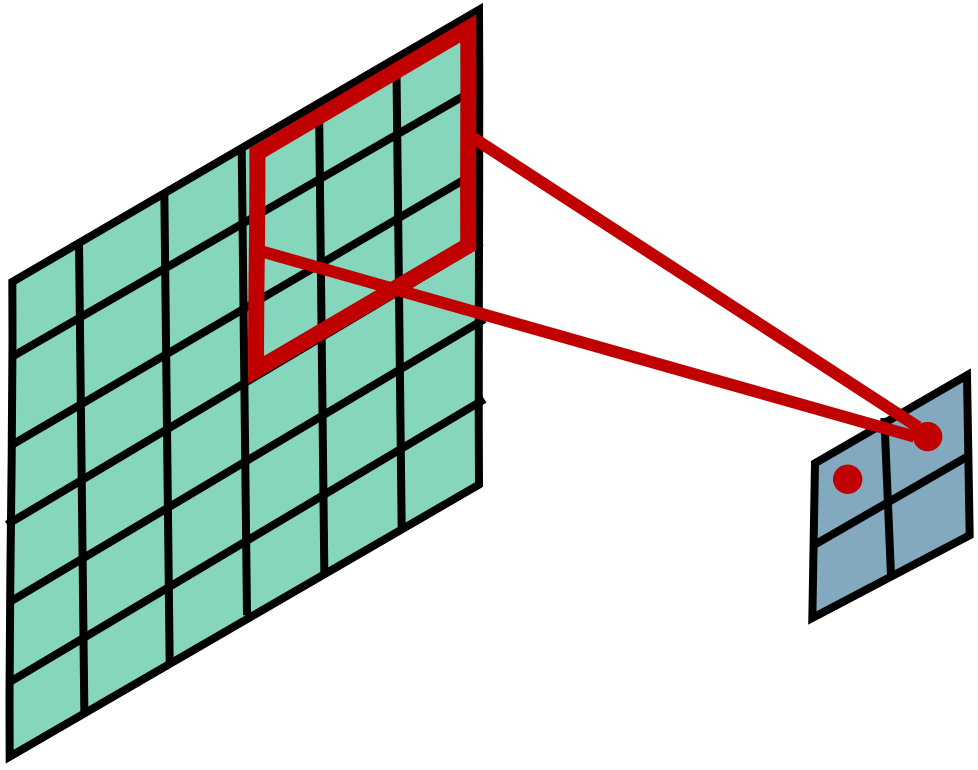
Stride = 1



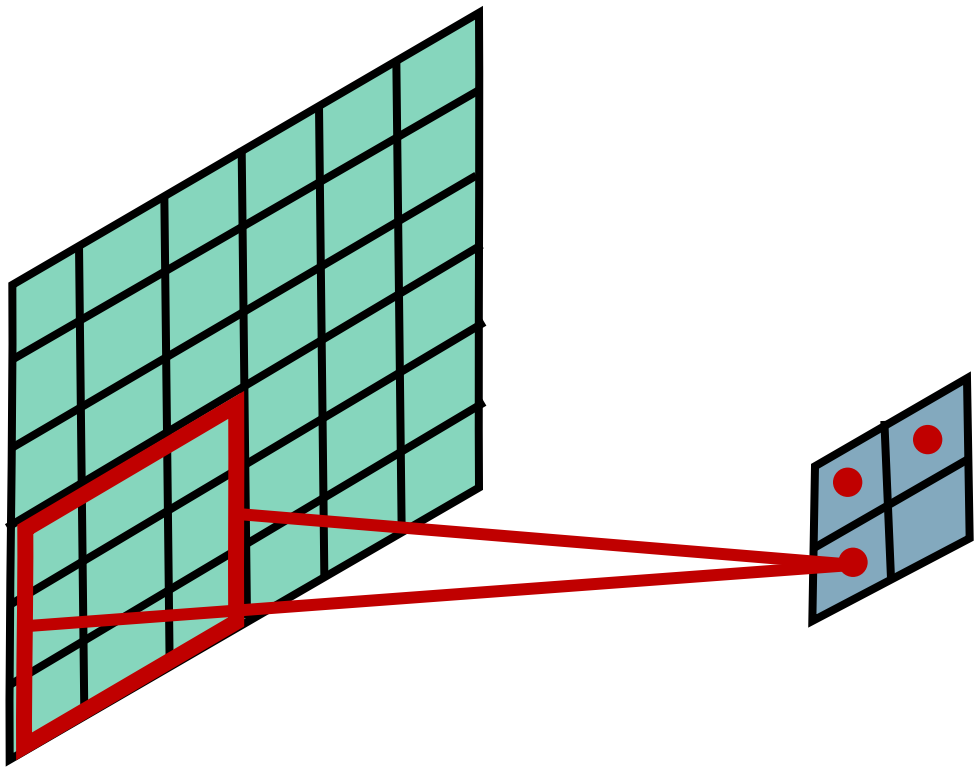
Stride = 3



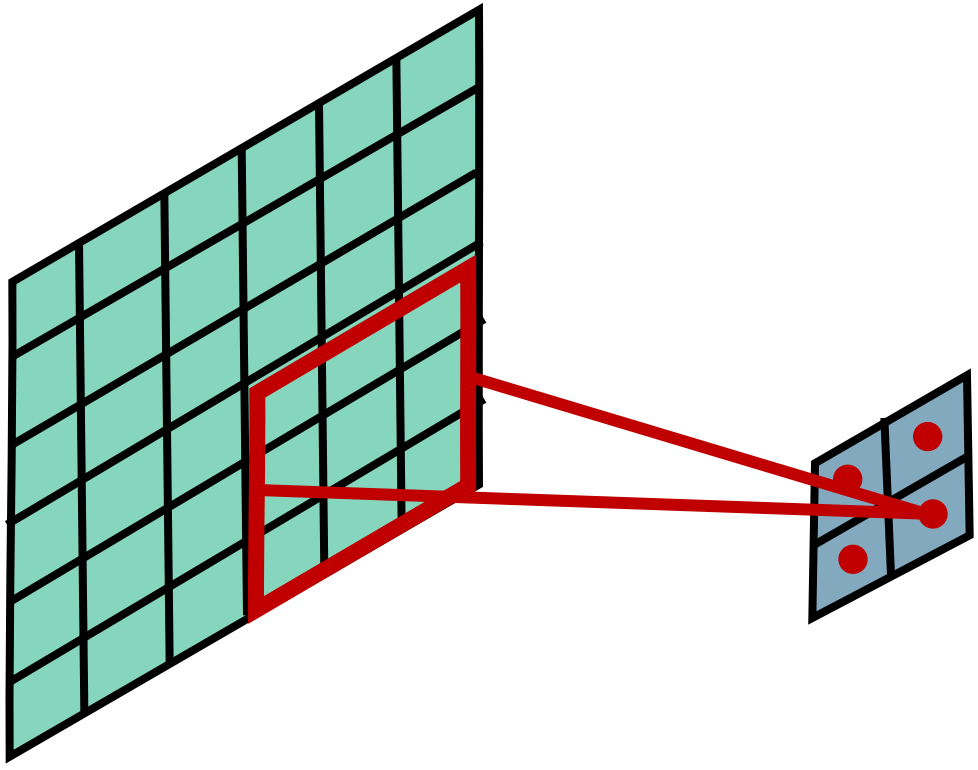
Stride = 3



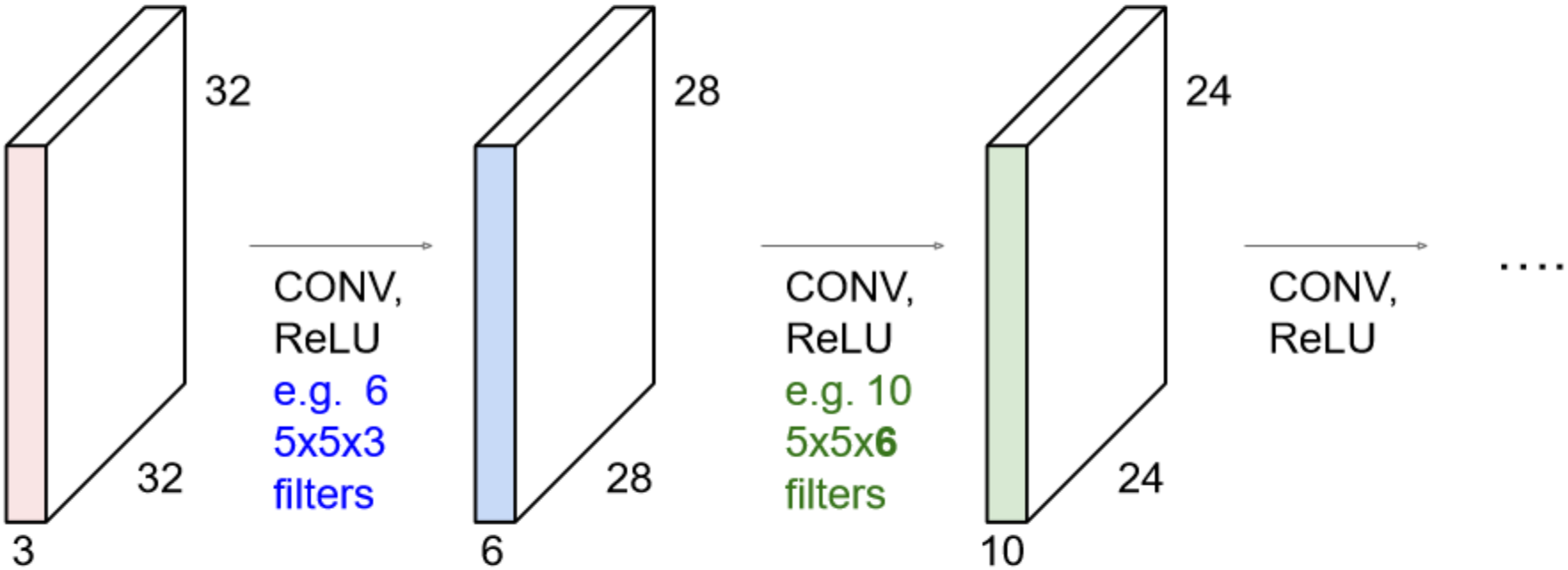
Stride = 3



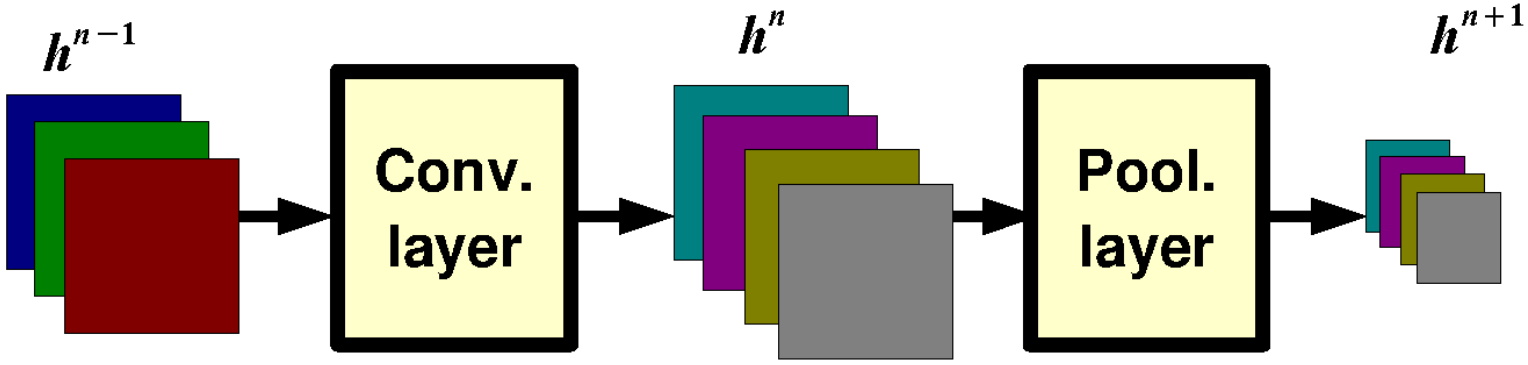
Stride = 3



Convolutions: More detail



Pooling Layer: Receptive Field Size



Pooling Layer: Examples

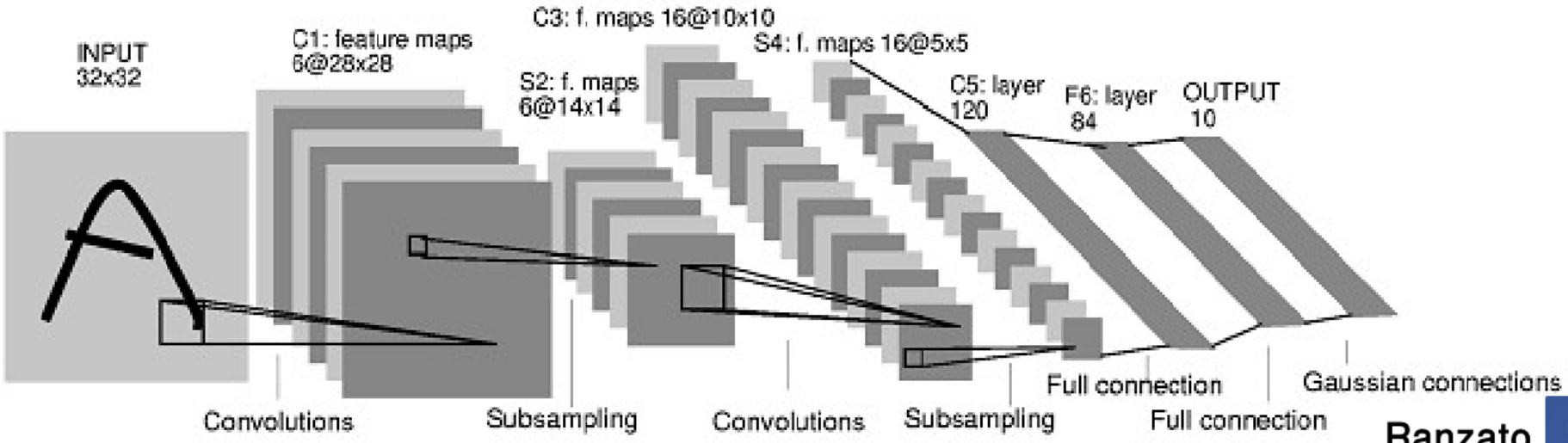
Max-pooling:

$$h_j^n(x, y) = max_{\bar{x} \in N(x), \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Average-pooling:

$$h_j^n(x, y) = 1/K \sum_{\bar{x} \in N(x), \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Yann LeCun's MNIST CNN architecture



LeNet-5

Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," Prof. IEEE, 1998.

A Common Architecture: AlexNet

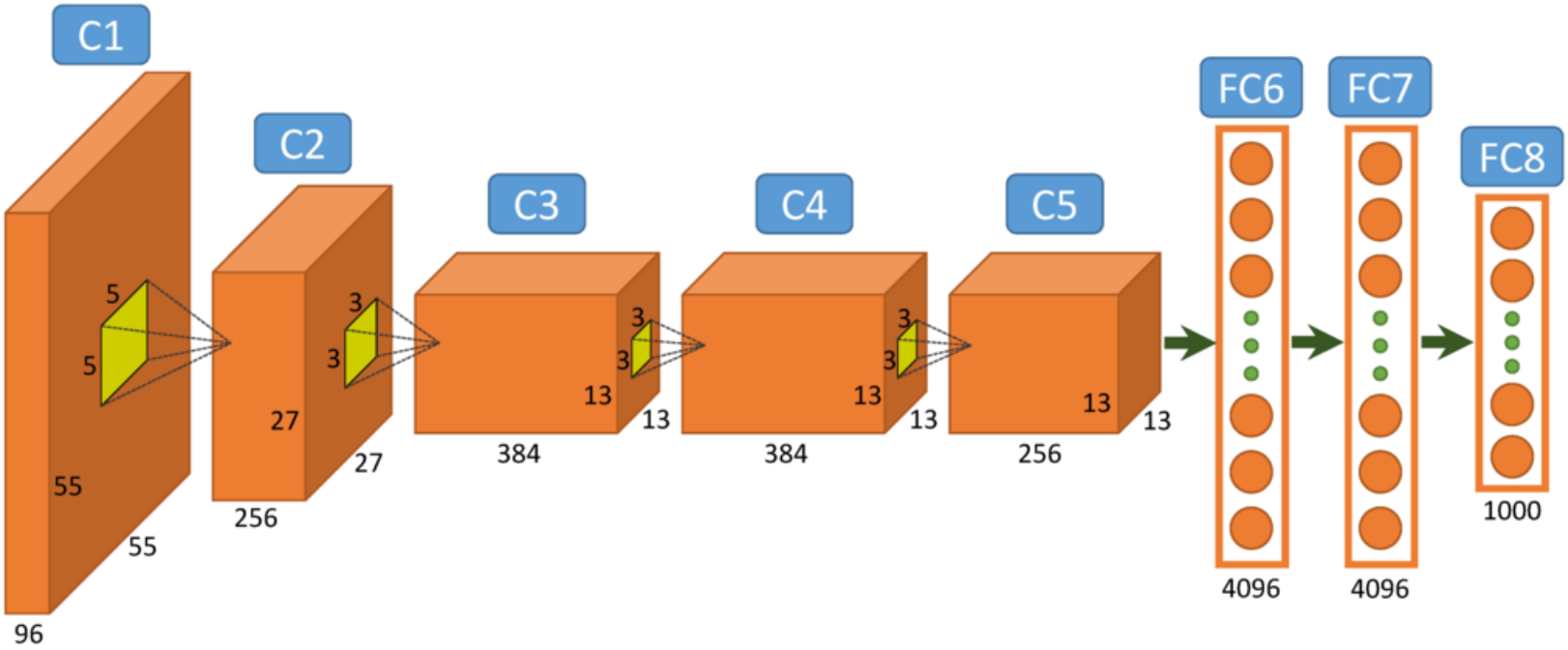
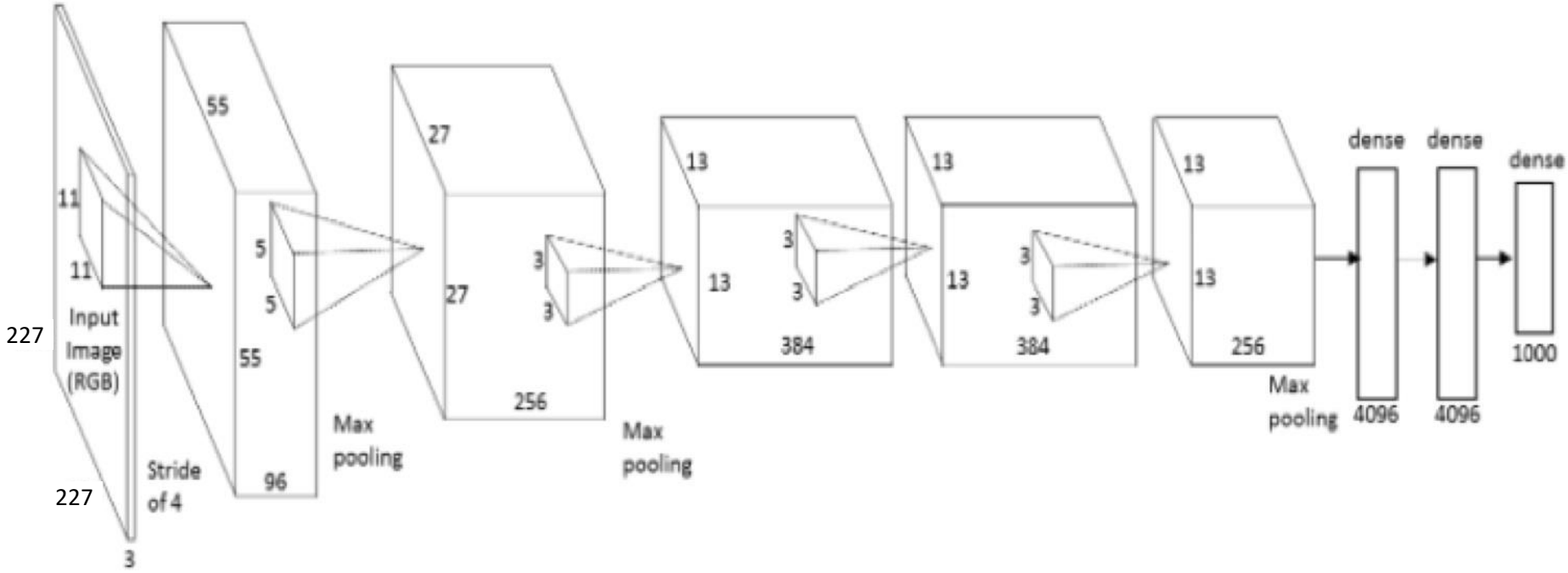


Figure from http://www.mdpi.com/2072-4292/7/11/14680/htm

[Krizhevsky et al. 2012]

AlexNet diagram

Input size
227 x 227 x 3



Conv 1
11 x 11 x 3
Stride 4
96 filters

Conv 2
5 x 5 x 48
Stride 1
256 filters

Conv 3
3 x 3 x 256
Stride 1
384 filters

Conv 4
3 x 3 x 192
Stride 1
384 filters

Conv 5
3 x 3 x 192
Stride 1
256 filters